

HANDBOOK

# Broadside

## Help and reference

Every tab of Broadside, control by control: what it does, what it will ask you first, and what it will not do behind your back. With a glossary for the terms and two reference tables for the keys and the warnings.

**Version 1.5.0** · October 2026

# Contents


1 Overview and getting started

2 Hosts

3 Templates

4 Terminals

5 Broadcast

6 PTY Broadcast

7 MultiTerminal

8 Skills

9 SFTP

10 Logs

11 Settings

12 Security

13 System requirements

14 Keyboard shortcuts

15 Warnings and messages

16 Glossary


This is the same help the app carries, printed as it stood at version 1.5.0. The copy inside Broadside is the one that is kept up to date: open **Help** in the app if this document is older than the version you are running.

SECTION 1

# Overview and getting started

Broadside is an SSH console built around one idea: send one command to many machines at once, while still giving each machine its own full interactive terminal when you need it. It also opens local Windows shells in the same window.

### The left rail

The buttons down the left side switch between the main areas of the app: **Hosts**, **Templates**, **Terminals**, **Broadcast**, **PTY Broadcast**, **MultiTerminal**, **Skills** and **SFTP** at the top, and **Logs**, **Settings**, **Help** and **About** at the bottom. The rail shrinks to icons on narrow windows, or with the collapse button at the top. Drag its right edge to resize it, or double-click that edge to reset the width. Broadside opens on **Hosts**; **Settings → Startup** picks a different first tab.

### Finding your way

- Hover almost any control and a short explanation appears in the status bar along the bottom of the window. The right-hand end of the bar counts what the page is showing. On **Terminals**, **16/20 sessions connected** means 20 tabs are open and 4 of them (the red ones) have no live connection.
- When nothing is hovered, the left-hand end shows the window size, the terminal font size and the app font size, and updates them as you resize or zoom. While the session timeout is on, it is shown there too (**Session timeout 30m**), so you always know it is set.
- Every message that pops up in the bottom corner (a toast) is also kept in **Logs → Toasts History**, so one that went by too fast can be read again.
- A spinner in the status bar with a short line such as **SFTP transfer in progress** means work is still running on another tab. It stays until that work finishes.
- The app version is shown next to the Broadside name at the top of the rail and in the About dialog.
- Switch between light and dark appearance in **Settings**, under **Appearance**.
- Press **F11** (or the maximize button on a terminal tab) to fill the whole window with a single terminal, and **F11** again to return. **Esc** does not come back out: it goes to the program in the terminal, because editors such as `vi` depend on it. **Alt+Left** / **Alt+Right** still change sessions while you are maximized.

SECTION 2

# Hosts

The **Hosts** tab is your address book of machines to connect to. Each host records how to reach it (address, port, username) and how to sign in.

### Adding and editing hosts

- Give each host a label, address, port and username. You can assign a color so the host is easy to spot across the app.
- Choose how to sign in: a password or an SSH key (with an optional passphrase). You can also store a sudo password for administrator commands.
- When you pick a key file, Broadside checks straight away whether it is protected by a passphrase. If it is, the passphrase becomes required, and a wrong one is refused before you save. This is the only place a key passphrase is asked for: a terminal never prompts for it.
- For your safety, saved passwords and passphrases are never shown again after you save them. You can replace a secret, but the app will not display the stored value. See the **Security section** for where these are kept.
- The eye beside any password, passphrase or passcode box shows what you have typed. Shown, it is in a fixed-width font with digits in blue and symbols in red, so `1`, `l` and `I`, or `0` and `O`, cannot be mixed up. The recovery code is coloured the same way.
- A host row can carry a small amber chip, most often naming a login shell Broadside cannot fully work with (fish, for example). Other pages show the same mark as an amber triangle beside the host. The host is never hidden, and terminals and broadcasts to it still work. **Warnings and messages** explains every chip, banner and error.
- **Shell integration**, at the foot of the host's form, controls a short setup line Broadside sends when a terminal opens. It lets Broadside see where each command starts and ends, which **MultiTerminal** and **Skills** rely on. **Auto**, the default, decides from what the host reports. **Always** is for a bash or zsh host that will not say what its shell is (a restricted account, for example). **Never** sends nothing and leaves the login screen exactly as the host draws it. Devices that greet you with a menu instead of a shell are detected and left alone automatically. A change applies to the next terminal you open on that host.
- On some hosts that setup line is saved in the shell's history, so the **Up** arrow finds it instead of your last command. **Settings > Security** can remove it for you; see the **Security section**.
- **Reconnect check** helps when you have just rebooted a machine. When a terminal on this host loses its connection, a pill on its **Reconnect** banner keeps checking whether the host is back: red while it is down, green once it answers, so you do not have to keep clicking **Reconnect** to find out.
  Right after you type `reboot` the pill may read **SSH still answering**, because the machine has not gone down yet. Wait for **SSH back**: that is the one that means you can reconnect.

The setting picks how often it checks (every 1 to 10 seconds, 3 by default), or **Disable** turns it off for this host. The check only looks for the host's SSH port answering. It never logs in, and it stops when you reconnect or close the tab.

### Importing many hosts

**Import hosts…** and **Export hosts…** are both in the **Import / Export** menu at the top of the page. **Import** loads a list of hosts from a csv file or an Excel .xlsx workbook (only its first sheet is read). Older .xls workbooks are not supported: save them as .xlsx or .csv first. The importer only reads connection details, never passwords. If a label is already in use for a different machine, the import keeps both by adding a numbered suffix (for example web becomes web-2).

Tip: to build that file without guessing the format, add one host here manually first, use **Export** to save the list to a CSV or spreadsheet, then fill in the rest of your hosts in that same file (keeping its column headers) and **Import** it back.

### Working with the list

- The **Status** column shows whether each host currently answers on the network.
- A colored dot marks hosts that have a live terminal connection open right now.
- Select several hosts and open a terminal for all of them at once. Double-click a host's row to open a terminal on just that one.
- Drag a host by the grip beside its checkbox to change its place in the list. The Terminals **+** menu and the broadcast lists use the same order, and a new host goes to the end.
- Click a column header to sort by it, again to reverse, and a third time to go back to your own order. Dragging a host while the list is sorted keeps the order you see as your own and turns the sort off.
- The **Find by label…** box at the top right narrows the table to hosts whose label matches what you type. It works together with the tag filter and clears with the **x**.
- You can hide columns you do not use from **Settings**, under **Appearance**. When a column is hidden, each row gets a small tag/label icon in its **Actions**; click it to see that host's values for the hidden columns.
- Drag a column's name in the header to move the column, or drag the thin strip on the header's right edge to resize it. The order is remembered across restarts. You can also reorder the columns in **Settings → Appearance**, where **Reset defaults** puts them back. The checkbox, colour swatch and **Actions** columns stay where they are.

### Tags

- Give a host one or more tags to group machines (for example prod, web, eu). Separate multiple tags with commas. Each tag shows as its own small chip in the **Tag** column; untagged hosts show a dash. As you type a tag on the host form, a dropdown suggests matching tags already used elsewhere, so spellings stay consistent.
- Click the **Tag** header to sort by the host's first tag (untagged hosts sink to the bottom, or rise to the top when you reverse the sort).

- The filter icon in the **Tag** header opens a checklist of every tag in use. Uncheck tags to hide the hosts that only carry them; a host stays visible while any of its tags is checked. The icon is highlighted while a filter is on, and the filter resets when you restart the app.
- Drag the separator on the right edge of the **Tag** column to resize it, or double-click that edge to fit it to the contents. The width is remembered.

SECTION 3

# Templates

A **template** is a saved arrangement of terminal sessions. Build it once, and one action opens the whole set: four servers and a local shell, in the order you want them, without opening five things by hand every morning.

The **Templates** tab on the left rail is where you build and edit templates; nothing launches from it. You open them from the **Hosts** tab, using the **Hosts** | **Templates** | **Profiles** switch at the top.

### Building one

- The rail on the left of the editor has four groups: your saved **hosts**, the **local shells** this machine has, your **custom terminals** (Git Bash and the like) and your saved **shell profiles**. Drag a row into a box, or click it to add it at the end.
- Sorting works within each group, so hosts, shells and profiles never get mixed together. A shell hidden in **Settings → Appearance** is hidden here too.
- Drag a box onto another to change the order, and use the small **x** on a box to take that session out. The number in the corner is the order it opens in.
- The same host can appear more than once.
- To keep a grid you already have open, press **Save as template** on the Terminals tab (beside **Grid Titles**). This tab opens with those sessions on the canvas, in tile order; name it and press **Create**. The button shows in Grid with two or more tiles, and only when no saved template already opens exactly those.

### The preview

The boxes are laid out with the same grid settings as the **Terminals** grid, so what you see is what launching will build. **Grid settings**, in the line under the canvas, changes them without leaving the page. There is one set of grid settings, so a change here changes Terminals too. Drop new sessions on the dashed **Drag a session here** strip.

A template saves only the sessions and their order, not a layout, so the same template looks right on a laptop or a wall of monitors.

### Names, and finding one again

- Two templates cannot share a name. Save a second “Morning check” and it is stored as **Morning check 02**.
- Changing the name of a saved template and pressing **Save** renames it; it does not make a copy. To copy one, use the **Duplicate** button on its row in the Hosts tab's Templates list. The copy is numbered like a clashing name, appears directly under the original, and never opens at startup.
- Closing the editor with the **x** shows **New template**, **Import** and the five templates you edited most recently, with a link to the full list on the Hosts tab.

- The server button beside the template menu, at the left of the editor's bar, goes straight to **Hosts → Templates**. Whatever is on the canvas is still there when you come back.
- Putting something else on the canvas while it has unsaved changes asks first: another template from the menu, **Edit** on the Hosts tab, **Edit** or **Duplicate** in the Terminals tab's **plus button** menu, **Save as template** on the Terminals tab, or an import. **Keep editing** leaves your work as it was.
- If you delete a template on the Hosts tab while it is open in the editor, nothing on the canvas is lost: **Create** saves it again as a new template, or **Discard** closes the editor.

### Opening one

- On the **Hosts** tab, switch to **Templates** and press the play button on a row, or double-click the row. It opens every session in the saved order and takes you to the Terminals tab.
- Or open one from the **Templates** section of the Terminals tab's **plus button** menu.
- To open several, tick them and press **Multi-template**. They open one after another, top of the list first, and a dialog shows that order before anything opens. Drag a row by the grip beside its checkbox to change the order.
- With terminals already open you are asked where the new tabs go: **New workspace** (a workspace of their own, named after the template), **add to the front** or **add to the end** of the workspace on screen, or **cancel**. Several templates with New workspace get one workspace each. Once there is more than one workspace, a line under the front and end buttons names the one they add to. Opening a template never closes anything.
- Anything the template names that this machine does not have (a deleted host, a removed WSL distro) is listed in that dialog and skipped. The rest still opens.

### Opening at startup

One template, at most, can open itself when Broadside starts. The setting is in the editor's top bar and has two on positions:

- **Open the tabs at startup** opens the tabs without connecting. Each one waits on its **Reconnect** button.
- **Open and connect at startup** also connects them, so live sessions start as soon as Broadside does. Saving this asks you to confirm.

Choosing it on one template takes it off whichever one had it. While a template opens at startup, Broadside starts on **Terminals** and **Settings → Startup** is greyed.

### Export and import

- An exported file lists each host by label, address, port and user name. It never contains credentials.
- Importing matches those against this PC's hosts. Anything that does not match arrives as a named box for you to point at a real host. Importing never creates or changes a host, and never sets a template to open at startup.

> - Local shells and profiles belong to one PC. They travel by name, so a template made elsewhere still tells you what it wanted when this PC does not have it.

SECTION 4

# Terminals

**Terminals** gives you full, interactive terminal sessions, one per tab. Each tab is a real shell running over a PTY, so editors, progress bars and anything that needs a live screen all work normally.

### Remote and local shells

- Open a remote shell from the **Hosts** tab, or from the **Hosts** section of the **plus button** menu at the end of the tab strip. The connection runs over SSH.
- The same **plus button** opens local Windows shells: Windows PowerShell, PowerShell 7 (pwsh), Command Prompt and any installed WSL distributions.
- While terminals are open, the row of controls at the top (**Tabs**, **Grid**, **View**, **Sessions** and the **plus button**) is tucked away behind a small tab at the top centre. Hover or click the tab to bring the row down; it goes back up once you have used it, and **Ctrl+Shift+T** brings it down with the keyboard in it. To keep it on screen, untick **Settings → Sessions → Controls row**.
- Under the **How many** box the menu has five sections: **Local shells**, **Hosts**, **Profiles**, **Templates** and **Serial console**. Click the arrow beside a section's name to roll it up; it stays rolled up until you open it again. Long lists scroll inside the menu.
- Drag a section by the grip at the left of its heading to put the first four in the order you like. The order is kept across restarts. **Serial console** always stays last.
- Click a template under **Templates** to open it. If tabs are already open, you are asked whether the new ones go at the front or the end. Hover a template for **edit**, **duplicate** and **delete**: edit and duplicate open it on the **Templates** tab. A duplicate arrives named with the next free number (**Name 02**) and is not saved until you save it there. Drag the grip to change the order, which is the same order as the Hosts tab's template list.
- **How many** applies to hosts as well: set it to three and one click opens three sessions. A host that already has a tab is marked **already open**, and picking it opens another. **New** beside the **Hosts** heading takes you to the form for adding a host.
- Rename a local shell tab with the **pencil** on the tab: type a name and press **Enter** (or **Esc** to cancel). An empty name puts the shell's own name back. The new name lasts until you close Broadside.
- A second tab on the same shell or host is numbered: **PowerShell**, then **PowerShell 02**, **PowerShell 03**. Moving tabs around never changes the numbers.

### Workspaces

- A workspace is a group of open tabs. The Terminals tab shows one workspace at a time; the terminals in the others keep running out of sight.

- Start one with **New workspace** in the **Sessions** menu, or **Move to a new workspace** on a tab's right-click menu. Once there are two, a row of workspaces appears above the tabs: click one to show it, and its **+** starts another.
- Anything you open lands in the workspace on screen. Move a tab to another by right-clicking it, or by dragging it onto that workspace in the row.
- Double-click a workspace to rename it. Right-click it for a colour or **Close workspace**, which closes every tab in it after asking. **Close all workspaces**, at the bottom of the **Sessions** menu, closes every tab in every workspace, also after asking.
- Opening a template while terminals are open offers **New workspace**, which puts its tabs in a workspace named after the template. A template opened into an empty, unnamed workspace gives it its name too.
- **Close all**, **Close others**, the Filter, the grid and the **Broadcast** rail only see the workspace on screen, so nothing is sent to a terminal you cannot see.
- A workspace that loses its last tab goes away. **Workspaces are not saved**: when Broadside closes they are gone. To keep one, use **Save as template**, and open that template into a new workspace next time.

### Serial consoles (COM ports)

- The **plus button** menu lists this PC's COM ports under **Serial console**. Picking one opens the console cable as an ordinary terminal tab, beside your SSH sessions. Where Windows knows the adapter's product name, it is shown under the port.
- The box beside each port is the **line speed** (baud). It starts at **9600**, the usual console speed; if your device wants another, such as **115200**, its documentation will say. The speed you use is remembered for that port.
- A new serial tab stays empty until the device says something. Press **Enter** to bring up its prompt. If you get garbage instead of text, the speed is probably wrong; it is shown on the tab (**COM3 @ 9600**).
- A COM port can be open in only one program at a time. If another program (PuTTY, Tera Term) has it, close that one and press **Try again**.
- **How many** does not apply to serial ports, and resizing the pane does not tell the device its new size.
- Serial tabs can be broadcast to from **PTY Broadcast** and the **Broadcast** rail. Every serial send asks for confirmation first.

### How local shells are detected (important)

- Broadside finds shells through your Windows PATH, so it does not matter where they are installed as long as the installer added them to PATH. A shell that is not on PATH (an unzipped portable copy, for example) will not appear in the list.
- Broadside reads PATH when it starts. If you install a new shell while the app is already running, restart Broadside for it to show up in the launcher.
- Hide shells you do not use in **Settings** under **Appearance**.

### Your own terminal applications

- Detection only finds the shells Broadside knows how to look for. For anything else, such as **Git Bash**, MSYS2 or Cygwin, open **Settings** and the **Custom terminals** section and add it yourself. **Add Git Bash** fills in the usual path and options for you.
- Give it a name (used in the launcher and as the tab title), the full path to the program (**Browse** finds it), any arguments it needs, and which command language it speaks. For Git Bash, the arguments `--login -i` give you its familiar coloured prompt and the full set of Unix tools.
- The program must be a full path to a real `.exe`; Broadside checks this when you save.
- The command language decides which **PTY Broadcast** tab the terminal joins, so a broadcast command always suits every shell it reaches. A Bash-like (posix) terminal joins the **WSL / POSIX** tab.
- A terminal you add appears in the **+** menu straight away, with no restart, and can be used by a saved shell profile like any other shell. Removing one takes it out of the launcher; a profile still pointing at it is flagged rather than silently broken.

### Tab colours

- A host's tabs and grid tiles wear the host's colour. Local tabs are coloured by the kind of shell (one colour each for PowerShell, Command Prompt, WSL, Bash-like terminals and serial), so you can see at a glance which command language you are about to type in. Change them, or set any to **None**, in **Settings** under **Custom terminals**; **Defaults** puts them all back.
- A **profile** can have its own colour, picked when you create it, so two profiles on the same shell can be told apart. Left alone, it uses its shell's colour.

### Shell profiles

- A profile is a saved local shell that opens the way you want it: a chosen shell, a working directory, and an optional command that runs the moment the shell is ready. Open the **plus button** and use the **Profiles** section at the bottom of the menu to create, launch, edit or delete them.
- **New** opens a short form: a name (which becomes the tab label), the shell, a **Working directory** (type it or use **Browse**; leave it empty to start in your home folder), an optional **Startup command** and an optional **Tab colour**.
- Launching a profile opens the shell in that folder and, once the prompt appears, types the startup command and presses Enter for you. For example, a profile can open PowerShell 7 in a project folder and start a command-line tool there in one click.
- Profiles honour the **How many** count. A profile with a startup command asks before opening more than one copy, since each copy runs the command again.
- Hover a profile in the launcher for **edit**, **duplicate** and **delete**. **Duplicate** puts a **(copy)** just below the original, which is the quick way to make profiles that differ by one setting. **Delete** needs a double-click and then a confirmation, so a stray click cannot remove one. Terminals already opened from a deleted profile keep running.
- Drag a profile by the grip at the left of its row to change its place in the list.

- **The Hosts tab has a fuller list.** Set the **Hosts** | **Templates** | **Profiles** switch at the top of the **Hosts** tab to **Profiles** for a table that shows each profile's folder and startup command. Everything the launcher does works there too, and you can also export a single profile, or just the ones you tick. The terminal button on a row, or a double-click on the row, opens that profile, and **Multi-profile** opens every ticked one, each in its own tab. The columns can be moved, resized and hidden like the Hosts table's (hide them in **Settings** → **Appearance**).
- **Export** saves all your profiles to a file, to carry them to another PC. **Import** shows every profile in a file, including its startup command in full, before anything is saved. **Read those commands**: a startup command runs every time the profile opens, so only import a file you trust. A profile whose name you already use comes in as a numbered copy.
- A profile whose shell is not on this PC (a removed WSL distribution, or a custom terminal you do not have) is shown struck through and will not launch. Hover it to see which shell it wants. Once you add that shell, **Edit** on the profile has it already picked, and you just save.
- Profiles never store passwords or keys. They are kept by **Settings** → **Reset everything** and included in backups. A restore brings them back with their startup commands empty, so type those again afterwards.

### How long a session stays open

- **Settings** → **Sessions** → **Keep alive** stops a firewall or router quietly dropping a connection you have left idle. It is on by default and applies to connections opened after you change it.
- **Settings** → **Sessions** → **Session timeout** closes an SSH terminal that has been idle for the time you set. It is **off** by default. Idle means nothing typed, nothing printed and no command running, so a long quiet command, an editor, `tmux` or `screen` keeps the session open; a job left running in the background does not. On a very basic shell (a plain `sh`, or a device's own menu) Broadside cannot tell whether a command is running, so only silence counts.
- A pane closed by the timeout keeps its output, says the timeout closed it, and offers **Reconnect**. Local shells and serial consoles are never timed out.

### Working with tabs

- Drag tabs to reorder them. Reordering never disconnects a session.
- A remote tab whose connection is down (still connecting, dropped or failed) shows its label in red; use **Reconnect** to bring it back. Local shell tabs never show this.
- A pane that is opening shows **Connecting**. If that takes more than a few seconds, a second line says whether the wait is on the network or on the host.
- **Alt+Right** and **Alt+Left** switch to the next and previous tab, even while you are typing in a terminal. To reach the tabs without a mouse, **Tab** into the tab strip, move along it with **Left** and **Right**, and press **Enter** to choose one.
- Maximize a single terminal to fill the window (also **F11**). Close one tab with its **x**, or all of them with **Close all** in the **Sessions** menu.

### Grid view

- The **Tabs** / **Grid** toggle above the tab strip switches between one terminal at a time and every terminal tiled at once. Tiles follow the order of the tabs and resize with the window. When there are too many to read comfortably, they keep a readable size and the grid scrolls. Each tile has its own maximize and close buttons.
- **In Grid, the tile under the mouse is the active one.** It is outlined in its host's colour, and it gets whatever you type, paste, search for or run from the shortcut menu. Passing over a tile does not select it; the pointer has to rest there for a moment (a click selects it straight away). Choosing a tab, using **Alt+Left** / **Alt+Right** or picking from **Sessions** selects a tile until you move the mouse onto a different one. While another app is in front, the mouse does not change the active tile; click a tile to come back to Broadside.
- **Short of room?** **Ctrl+Shift+G** gives the grid the whole screen (see below). **Grid Titles** (the eye) hides every tile's title bar. The grid button on a tab takes that terminal out of the grid without closing it (the button turns amber; click it again, or **Unhide all** in the **Sessions** menu, to bring it back). **F11** gives one terminal the whole window.
- **Save as template**, next to **Grid Titles**, turns the tiles on screen into a new template on the Templates tab. It shows with two or more tiles, and hides while the grid matches a template you already saved.
- **Fullscreen** (**Ctrl+Shift+G**) gives the grid the whole screen. The toolbar, tabs, sidebar and status bar slide away behind a small tab in the middle of the top edge; hover or click that tab to bring them back, and they hide again once you have used one. **Ctrl+Shift+T** also brings them down and moves the keyboard into them: **Tab** moves between controls and **Esc** goes back to the terminal. Press **Ctrl+Shift+G** again to leave (closing the last terminal also leaves). **Esc** does not leave fullscreen, so it still reaches programs such as `vi`.
- Drag the right or bottom edge of a tile (or its corner) to resize **every** tile at once. A dragged size is forgotten when Broadside closes, unless you untick **Strict** in **Settings → Grid**, which saves it. **Reset grid** goes back to tiles that fit the window.
- To resize tiles one at a time, untick **Resize tiles together** in the **View** menu (Grid only). A drag then changes only that tile, and the others wrap around it at the size they had, so rows can be ragged. These sizes are forgotten when Broadside closes. **Reset grid** clears them, and ticking the option again lines the tiles back up.
- To fix the layout, open **Settings → Grid**. Leave everything on **Auto**, or set **Columns across** and the tile **Item width** / **Item height** in characters. Tiles too wide for the window scroll sideways rather than shrink. One or two terminals always fill the space whatever these say, and a column count larger than the number of open terminals is reduced to fit. Changes apply the next time you open the Terminals page. The **Templates** editor uses the same settings.
- The collapsed **Advanced** part of **Settings → Grid** sets the number a field starts at when you untick its **Auto** (3 columns and 80x24 unless you change it).
- **Short last row** in **Settings → Grid** decides what happens when the terminals do not fill the last row: **Stretch the row** (the default) widens the tiles on it, **Tall tile in the last column** grows the tile above the gap down into it, and **Leave the gap** keeps every tile the same size.

A width or height you have set yourself always wins, and the gap stays.

### The broadcast rail

- The **Broadcast** button opens a rail down the left side: pick a shell language, tick the sessions you want, type a command at the bottom, and it is typed into all of them at once. Pair it with **Grid** to watch them all answer.
- It works on one shell language at a time, so a command always suits every session it reaches. The safety checks are the same as on the **PTY Broadcast** tab: a destructive command asks before going to SSH hosts, and every local-shell send is confirmed.
- **Up** and **Down** recall commands sent from any of the broadcast pages. **Clear history** clears the recall on this rail only, and the **Logs** record is left as it was.
- A disconnected session stays in the list in red, but cannot be ticked until you reconnect it.

### Copy, paste and find

- **Copy** by selecting text and right-clicking, or with **Ctrl+Shift+C**. Selecting on its own copies nothing. Double-click selects a word and triple-click a line. In a program that uses the mouse itself (tmux or vim with the mouse on), hold **Shift** while you drag.
- **Paste** with a **right-click** when nothing is selected, or with **Ctrl+Shift+V**. Plain **Ctrl+C** and **Ctrl+V** go to the shell as usual, so Ctrl+C still stops a running command. In Grid, a paste goes to the outlined tile. A toast confirms each copy and paste, and says how many lines a multi-line paste contained.
- In a program that uses the mouse itself (tmux, vim, Claude Code), a right-click goes to the program instead. Hold **Shift** and right-click to copy or paste there. To make a right-click always copy or paste, turn off **Settings → Sessions → Right-click**.
- Text copied from Windows programs pastes with single line breaks, not double-spaced. In bash, zsh and most editors a multi-line paste arrives as one block and waits for you to press **Enter**; a shell without that support runs each line as it arrives.
- **Find** with **Ctrl+F**. Matches are highlighted and counted (**3 of 17**); the arrows, or **Enter** and **Shift+Enter**, move between them. The toggles beside the box are regular expression, match case and whole word. **Esc** closes the bar.
- **Text too small?** Hold **Ctrl** and turn the mouse wheel over a terminal to zoom every terminal in or out. The size is saved in **Settings** under **Appearance**. If a program you use needs Ctrl and the wheel for itself, tick **Strict** beside the terminal font size there to turn the zoom off.
- **Scrollback** is 5,000 lines per pane by default, and older lines are dropped. Change it with **Terminal scrollback** in **Settings** under **Appearance**. More lines use more memory (5,000 is about 12 MB per pane), and lowering it drops lines above the new limit straight away.
- Every key Broadside takes is listed in **Keyboard shortcuts**.

### The Sessions and View menus

- **Sessions**, at the top right, lists every terminal: pick one to jump to it. It also holds the bulk actions, each shown only when there is something for it to do: **Reconnect all** and **Close failed** for panes whose connection failed or dropped, **Disconnect all** to end every session but keep the tabs, **Unhide all**, **Close others** and **Close all**.
- The small unplug button on each row of the **Sessions** menu ends that one session but keeps its tab and output, as if you had typed `exit`. The pane then offers **Reconnect**.
- Right-click a tab for the same actions as its icons, plus **Open another session** (a new session on the same host or shell, placed next to it) and **Close others**. When the other tabs are of mixed kinds you also get entries such as **Close other SSH** or **Close other Command Prompts**. Tabs hidden by the **Filter** are never closed this way.
- **Confirmations.** Bulk actions ask first when they would affect more than five sessions. Change that number (1 to 11) with **Ask before acting on more than** in **Settings** under **Performance**. **Close all** always asks, **Close others** asks whenever it would close two or more, and a profile with a startup command asks before opening more than one copy.
- After a reconnect, the old session's output stays in the scrollback above a line showing when it ended, and the new session starts on a clean screen.
- The **View** menu holds the display options: fullscreen, **Full tab labels**, **Tab bar**, **Filter**, flipping two terminals between side by side and stacked, **Reset grid** and **Reset display**. An entry that cannot be used right now is greyed; hover it and the status bar says why.
- **View** > **Filter** shows or hides whole kinds of session (SSH, each local shell, serial consoles). Hidden sessions keep running, **Alt+Left** / **Alt+Right** skip them, and the **View** button is marked while anything is hidden. A kind comes back by itself when you open or jump to one of its sessions, and every kind is shown again when Broadside restarts.
- **View** > **Tab bar** (Grid only) hides the tab strip to give its row to the tiles. The strip always comes back in **Tabs** view.

### Full-screen prompts, and a stuck one

Box-drawing prompts, such as the questions an upgrade asks, work as normal: arrow keys, **Tab** and **Enter** all reach them.

If one ignores your keys and they show up as `^[[A` or `^[` instead, the prompt was started by a script rather than from your terminal and cannot hear you. Press **Ctrl+C**, then run the command yourself, or put `DEBIAN_FRONTEND=noninteractive NEEDRESTART_MODE=a` in front of an `apt` command so those prompts do not appear at all.

**Reset display**, in the **View** menu, clears a garbled pane (including its scrollback) and redraws the prompt. It never interrupts a running command; if one is still going, the pane stays blank until it finishes. In a **Command Prompt** tab, press **Enter** afterwards for a fresh prompt.

### Shortcut commands

- The **Shortcut command** dropdown runs a saved command in the active tab. Each shortcut has a scope: **SSH / WSL (Linux)**, **Command Prompt / PowerShell**, **Both** for commands that work in either (for example whoami), or **Serial console (COM port)**.
- The dropdown only offers shortcuts that suit the current tab, so a Linux command is never offered in a Windows shell. WSL tabs use the Linux ones.
- Serial tabs only get **Serial console** shortcuts (**Both** does not include them), and none come built in, since every device is different. Add your own with **Manage shortcuts…** in the dropdown, or in **Settings** under Shortcut commands.

SECTION 5

# Broadcast

**Broadcast** sends one command to many hosts at the same time and gathers each host's output in one place. It is the fastest way to run the same one-shot command everywhere.

### How it runs

- Pick the hosts, type a command, and send. Each host runs it over an exec-channel and returns its output, which is grouped per host.
- Because the exec channel is not interactive, **Broadcast** is best for commands that finish on their own and print a result. For anything that prompts you or needs a live screen, use a **Terminals** tab or **PTY Broadcast** instead.
- A per-command time limit stops hosts that take too long. The output area keeps your history and follows new output as it arrives, and you can hide the per-host headers if you prefer a compact view.
- Hover a host in the rail for a **terminal** icon that opens a new terminal to that host. Hover a block of output and the host it came from lights up in the rail.

### Choosing and filtering hosts

- Pick targets in the host rail on the left. It collapses to color dots with the arrow button, has a **Sort** dropdown and a **Select all** box, and its collapse state is remembered. No hosts are selected by default; you choose the targets for each broadcast.
- Two filters narrow the rail: a **Find by label…** box that matches host labels as you type, and a **Filter tags** dropdown that works like the tag filter on the **Hosts** table. The two combine, and both reset when you restart the app.
- A host the filter hides is also unticked, and stays unticked when you clear the filter, so a hidden host can never be swept into a broadcast. **Select all** only picks visible hosts. The same rail filter is on **PTY Broadcast**, **MultiTerminal** and **SFTP Broadcast**.

### Safety

A destructive-command guard can warn you before sending commands that look dangerous, and ask you to confirm first. You configure which patterns count as destructive in **Settings**.

SECTION 6

# PTY Broadcast

**PTY Broadcast** types one command into every open interactive terminal at once. Unlike plain **Broadcast**, it drives the live shells you already have running in the **Terminals** tab.

### When to use it

- Use it when you want the same keystrokes delivered into several live sessions, for example to start the same interactive program on many hosts or local shells.
- It types into real shells exactly as if you typed in each tab, so there is no completion signal and no time limit. Use **Broadcast** for one-shot commands that return a result, and **PTY Broadcast** for interactive ones.

### One tab per shell language

- The page is split into fixed tabs: **SSH**, **PowerShell**, **Command Prompt**, **WSL** and **Serial consoles**. Each open terminal shows up under its own tab, so a single command is always valid for the shells it reaches. A small count on each tab shows how many of that kind are open.
- Take most care on **Serial consoles**: a line typed there goes into several pieces of network hardware at once, with no undo. Every send outside the **SSH** tab asks first.
- The **SSH** tab has the same **Find by label…** box and **Filter tags** dropdown as **Broadcast**, and keeps a history of past runs.
- A session that has disconnected, or a local shell that has exited, stays in the rail with a red dot but cannot be ticked.
- Hover a row in the rail for a **terminal** icon that takes you to that exact tab on **Terminals**. Hover a row of the run report and its session lights up in the rail.
- **Up** and **Down** recall commands sent from any of the broadcast pages. **Clear command history** clears the recall on this page only; the **Logs** record is left as it was.

### Local shells (PowerShell, Command Prompt, WSL)

- The three local tabs fan a command out to the shells running on *this machine*. A persistent yellow banner marks the danger: the command runs in every checked shell immediately, with no undo.
- Every local send asks you to confirm first, listing the exact command and the shells it will run in. Cancelling sends nothing.
- The report for local runs is cleared when you close Broadside.

SECTION 7

# MultiTerminal

**MultiTerminal** runs a command across the terminals you choose and shows every host's output together, each tinted with that host's color so you can compare results at a glance.

### How it works

- Pick which open sessions to target in the host rail on the left, type a command in the composer, and send. The rail collapses to color dots with the arrow button.
- The rail carries the same **Find by label…** box and **Filter tags** dropdown as **Broadcast**. Only connected, unfiltered sessions are selectable, and hiding a session with the filter unchecks it.
- Each command produces a block in the log, labeled with the command that ran. You can delete an individual block to keep the view tidy.
- Two sessions on the same host are listed separately and numbered the way their tabs are (`web-01`, `web-01 02`). A note above the composer tells you when two of the sessions you have ticked are on the same machine.
- Turn on **Show Manual Activity** to also capture commands you type by hand in a **Terminals** tab, for the hosts ticked in the rail. While it is on, those tabs show a small **layers** icon so you know they are being recorded.

### Block headers

- With **Headers** on, each block has a header you can click to collapse or expand its output. Hover the header for a **copy** icon, which copies that block's output.
- With **Headers** off, blocks show output only, with no header to collapse or copy from.
- Hover a block and the session that produced it lights up in the rail. Hover that row for a **terminal** icon that takes you to its tab.

### Rail actions

- **Clear results** empties the block log (and the saved history). **Clear command history** clears the Up/Down recall on this page only. **Close all terminals** closes every open terminal after a confirmation.

SECTION 8

# Skills

A **skill** is a saved sequence of steps that Broadside runs against every host you check, in a real interactive shell it opens for each one. Because it is a real shell rather than a one-shot command, a skill can become root and stay root, answer questions a program asks, and drive things that repaint the screen, while you watch each host's terminal live.

### The five kinds of step

- **run** sends a command and waits for it to finish, then branches on whether it succeeded.
- **wait for** watches the output for a pattern, optionally sending a reply the moment it appears. This is how you answer a prompt: wait for `Password:`, send the password.
- **send** types something with no waiting, for a keystroke a program is already sitting there expecting.
- **hold** pauses for a fixed number of seconds, sending nothing, so a status screen has time to settle.
- **post to the skill log** writes a line into that host's run log and carries on. It does nothing on the host.

### A skill that explains itself

**Post to the skill log** lets a skill record which way it went, for example *already on the current kernel, nothing to do*. Pick a **Log type** (**Note**, **Warning** or **Error**), write the message (inputs work here), and say where to go next.

An **Error** whose **Then** is **stop** ends that host as a failure with your message, which is how a skill turns a host away in its own words. An **Error** that goes anywhere else is only logged, and the skill carries on.

### Patterns

A **wait for** pattern is a regular expression tested against the output with the colour codes stripped out. Its anchors are line anchors, so `^Ready$` means a line that says Ready, not the whole screen.

### Interactive steps

Normally a **run** step knows the command finished because the shell comes back to a prompt. Some commands never do: `sudo -i` opens a nested shell, and an installer stops on a question. Tick **Interactive** on those, and the step moves on once the output goes quiet instead. You then drive the rest with **wait for** and **send**.

### Branching

- Every step says where to go next. **next** means the following step in the list, so a straight-through skill needs no wiring at all. **stop** finishes that host successfully.
- A **run** step branches on the command's exit code by default: one target if it worked, another if it did not.
- Tick **branch on the output** instead to decide on what the command printed. Give it a pattern and two targets, and the output test outranks the exit code entirely. Useful for a command that reports what it found without failing, such as `systemctl is-active`.

### Timeouts

Each waiting step has a time budget. When it runs out the default is to **pause and wait for you**: that host goes yellow and you choose to wait again, treat the step as done and carry on, or stop the host. Nothing is ever quietly reported as finished because a timeout expired.

You can instead choose to fail (which ends the host for a **Wait for output** step, or takes the **if it fails** branch for a **Run a command** step), or **go to a step** of your choosing, such as a retry or a recovery path.

### Inputs

Declare inputs to avoid hard-coding values into commands. Write `{{service}}` in a step and Broadside asks you for **service** just before the run. Go templates such as `docker ps --format '{{.Names}}'` pass through untouched, so they are safe to use.

### The flow map

- Clicking a skill's name opens its **overview**: what it does, the inputs it asks for, and a diagram of how its steps connect. Clicking the pencil skips straight to the editor instead.
- Click any step in the diagram to see its settings. The zoom controls sit at the top right; clicking the percentage returns to 100%.
- Amber warnings above the diagram point out a step nothing leads to, which will never run, and a loop with no way out, which will hit the repeat limit and fail the host. Loops are drawn as dashed amber arrows running back up the left. Warnings never stop you saving.

### Runaway loops

A step that loops back is a fine way to wait until something is ready, so Broadside limits loops rather than forbidding them. **Stop a host after a step runs N times** in the editor sets how many times any one step may run on a host (100 by default, which is also the maximum).

**Let steps repeat without limit** removes the limit, and it is dangerous: use it only if you truly know what you are doing. A skill that loops then runs until you stop it with **Stop host** or **Emergency stop**, repeating whatever it does (writing files, installing things) on every pass. It is off by default, and the flow map and the run confirmation both warn you when it is on.

### Running one

- Check hosts in the rail, then press play on the skill. Steps that match a **guard rule** need the same typed confirmation a dangerous broadcast does.
- Each host gets its own live terminal pane. A host that pauses shows its options there. **Stop host** ends one host and leaves the rest running; the **Emergency stop** button at the foot of the rail ends all of them at once, which can leave a host mid-upgrade.
- **Taking a host over.** While a skill is running, typing into its pane does nothing. Press **Take over** in the pane's header: the skill finishes the step it is on (the button reads **After this step…** meanwhile) and then hands you the shell. The other hosts carry on.
- When you are done, **Resume** hands the host back to the skill at the next step, or **Stop host** ends it. Anything you left half-typed is cleared first.
- A host that pauses *on its own*, because a step timed out, offers the same choices plus **Skip step**, which treats that step as done and carries on.
- Each pane's header names the current step, with its countdown. **Log** in the header shows what the skill decided for that host: each step, what matched, what was sent, what timed out and which branch it took.
- **Rerun**, beside **Close run**, closes the finished run and takes you back to the confirmation to start it again.

### Keeping the shell afterwards

A skill normally closes each shell when it is done. Turn on **Allow transfer** and the shell stays open instead, so you can press **To terminal** and carry on by hand in a **Terminals** tab, with the run's scrollback intact. Broadside checks whether that shell is root and warns you before you leave a privileged one open. Closing the run closes any it still holds.

### Building one

- Steps in the editor fold down to one line each; click a step to open it, or use **Expand all**. The step you last clicked is tinted, so you can follow it as you move it with the arrows.
- **Renumber** renames the ids to s1, s2, s3 in their current order and rewrites every branch to match, so the flow is unchanged. Use it to tidy up after reordering.
- Amber notes below the steps are advice, not errors: they never stop you saving or running. Only the red list does that.
- **Save** saves and keeps you in the editor; **Save and close** does both. **Cancel** leaves without saving, and asks first if there are unsaved changes.

### The rail

- Each host has a status dot, with a key to the colours under the list. **Green**: a terminal is open to it. **Dark green**: its SSH port answered the last check. **Amber**: it did not. **Red**: not being checked (no terminal open, and checking is turned off for it on the Hosts page). **Grey**: no answer yet. Hosts are checked every 15 seconds while this page is open.

- **Sort: Online first** puts the hosts you can use right now at the top: those with a terminal open first, then those that answered.
- The panel icon at the top of the rail collapses it, to give the editor more room.

### Organising them

- The arrows on each skill in the rail move it up and down the list. The order is yours and it is remembered.
- The card above the list pins one skill to the top of the rail, so the one you use most is always in reach. Choose it from the dropdown in the card's title, or double-click a skill in the list. Pinning a skill opens its flow map, and the branching icon on the card brings the map back later. A pinned skill is not repeated in the list below.
- A faint green tint marks the skill shown on the right.
- **Clone** starts a new skill from a copy of an existing one, named **(clone)**, which is the way to build a variant of something that works. It opens in the editor unsaved, so nothing is written until you press **Create skill**, and the original is untouched whatever you do next.
- **Export** writes a skill to a `.json` file, and **Import** reads one back, showing you its steps before anything is saved. Files carry the definition only, never credentials.
- A skill that uses newer features cannot be imported into an older version of Broadside; the message says it was *made by a newer Broadside*. Update the copy you are importing into.

### What a skill needs from the host

Skills need a **bash**, **zsh** or **sh** login shell. A host with any other shell carries an amber chip on the **Hosts** tab and in the list here. It can still be ticked, but it reports a failure when the run starts, and the skill carries on with the other hosts.

### What you see in the pane

The pane shows your commands as you wrote them and what the host printed. On some hosts you may also see a `printf` added to each command and a line like `__bsdone_0__` under it. That is harmless: it is how Broadside tells when a command has finished on a shell it cannot follow otherwise (one that is not bash or zsh, a host with shell integration set to **never**, or a nested shell).

### What a run leaves behind

Out of the box, in **Logs** → **Skills**: one entry per host per run, with the exit codes and the timings, and as much of the step-by-step as you ask for. Your host's own shell history is left alone, because Broadside asks each shell it opens to stop recording for the duration.

If you would rather the commands stayed in the host's history, where an **Up** arrow finds them, that is **Settings** → **Skills** → **Where to log**. You can have either or both.

SECTION 9

# SFTP

**SFTP** browses and moves files over the same secure SSH connection as the rest of the app (see sftp). It has two tabs: **Commander**, a two-pane file manager for a single host, and **Broadcast**, which sends or pulls one path across many hosts at once.

### Commander (one host)

- A two-pane view: your PC on the left, the remote host on the right. Click an item to select it; double-click a folder, or press **Enter**, to open it. **Backspace** goes up a folder and the arrow keys move the selection.
- Type the start of a name to jump to it: **h** selects the first item starting with h, and **he** typed quickly the first starting with he. If nothing starts with it, the first name that contains it is picked. Pressing the same letter again moves to the next match.
- Transfer by dragging an item from one side to the other. Folders copy recursively, with a progress bar and a running byte count.
- Select several items as in Windows Explorer (**Ctrl+click** for individual files, **Shift+click** for a range), then drag any of them to transfer the whole selection.
- Up to four transfer at a time and the rest wait. Each running transfer, and each folder still being scanned, gets a row in the bar along the bottom with its own **Cancel**; **Cancel all** also clears whatever is waiting.
- A folder over 100 files or 100 MB asks before it starts. Dragging several large folders asks about each one in turn.
- When everything in one drag has finished, a single message sums it up. Anything that failed or was cancelled gets its own message.
- Cancelling keeps the files that had already finished and deletes the one that was part-way through, along with any folder it left empty. A failed transfer does the same. A file that was being overwritten is not restored: cancelling leaves neither the old copy nor the new one. To keep files that are already there, set **When a file already exists** to **Skip existing** before you drag.
- Several at once helps most on a distant host; on your own network it is barely faster.
- You can make a new folder on either side, and rename or delete items. Local deletes go to the Windows **Recycle Bin** so they are recoverable. A remote folder must be empty before it can be deleted.
- Rename with the pencil on a row, or select it and press **F2**. A rename never replaces an item that already has the new name. On the host side, rename and delete are unavailable while a transfer is running.
- The terminal button beside **Disconnect** opens a new terminal to the connected host and takes you to the Terminals tab. The SFTP session stays open.

### Broadcast (many hosts)

- The **PUT / GET** toggle sets the direction: **PUT** sends one local file or folder to every selected host; **GET** pulls one remote path from every host into its own subfolder on your PC (one folder per host label, so files from different hosts never collide). Each host gets its own progress bar, and a few hosts transfer at a time while the rest wait their turn.
- On **GET**, the remote path may end in a wildcard (***** or **?**) to pull several files at once, for example **/home/user/*.log** fetches every matching file from each host. Only the last part of the path is matched; the folders leading up to it are taken literally.
- Because a broadcast write can overwrite data on many machines at once, it is gated: an amber banner explains the risk and you must type **CONFIRM** before *every* run. By default no hosts are selected, and the rail has the same tag + label filter as **Broadcast**.
- Point at a host in the rail and a terminal button appears at the end of its row: it opens a new terminal to that host and takes you to the Terminals tab.
- **Create path if it doesn't exist** (PUT only) makes the destination directory on each host when it is missing; it is a session-only toggle that resets on restart.
- Hosts you have not connected to before are gathered into one **trust** dialog after the run (see TOFU), and trusting them retries those hosts. A host whose key has *changed* is refused.

### When a file already exists

The selector on the tab row applies to *both* tabs: **Overwrite all** always replaces, **Newer only** replaces only when the source is newer, and **Skip existing** leaves same-named files untouched. It applies to every transfer, a single file as well as each file in a folder, and is remembered across restarts. A file it leaves alone is not sent at all, and the summary message says it was skipped.

SECTION 10

# Logs

The **Logs** tab is your record of what happened: saved terminal sessions, an audit log of security-relevant actions, your past commands, what your skill runs did, every toast the app has shown, and the errors.

> **What you can find**
>
> - Saved sessions let you reopen and read the captured output of an earlier terminal session.
> - The audit log records important actions over time. You can switch it between compact and pretty-printed views, and turn audit logging on or off in **Settings**.
> - Command history lists commands you have run, and every view has a search box.
> - **Skills** holds one entry per host per skill run: what ran, how it ended, and how long it took. How much detail each entry carries, whether entries are written at all, and whether they are shown as readable text or raw JSON are set in **Settings** → **Skills**. **Export** always writes JSON.
> - **Toasts History** lists the last 500 toasts (the messages that pop up in the bottom corner), newest first, with the time and whether each was a success, a warning or an error. It is not affected by a restore and is not part of a backup. **Clear history** empties it and **Export** saves it to a file.

SECTION 11

# Settings

Settings is where you tune how Broadside behaves. The list on the left jumps to each section, and the search box above it narrows the page to what you type.

> **What you can change**
>
> - **Startup**: which tab Broadside opens on (**Hosts** by default). It is greyed while a template is set to open at startup, because that always opens on **Terminals**.
> - **Performance**: how many hosts the app talks to at once.
> - **Network probe**: how the app checks whether hosts are reachable for the **Status** column.
> - **Destructive command guard**: the patterns that trigger a confirmation before a risky command is sent.
> - **Shortcut commands**: reusable commands for the dropdown on the Terminals and Broadcast pages, each for Linux, Windows, both, or serial consoles. See **Terminals**.
> - **Sessions**: the idle session timeout (off by default), keep alive (on by default), and whether a right-click goes to a program that uses the mouse (on by default). See **Terminals**.
> - **Appearance**: light or dark theme, the terminal font and its size, the size of the app's text, which columns the **Hosts** and **Profiles** tables show and in what order, and which local shells appear in the **plus button** menu. Changes apply as you make them, except the two font sizes, which need **Save font sizes**.
>   **A font missing from the list?** **Terminal font** only lists installed fixed-width fonts, because a terminal needs every character to be the same width. Look for a version with *Mono* in its name. A font installed while Broadside is running appears after a restart.
> - **Skills**: what a skill run records, and where.
>   **Where to log**: **Skill log only** (recommended) records runs in **Logs** → **Skills** and keeps the commands out of each host's shell history. **Remote history only** leaves them in the host's history, where **Up** finds them, and records nothing here. **Both** does both.
>   **Logging type**: **Skill summary** (one entry per host per run), **Itemized** (adds a line per step with its exit code), **Run panel log** (the full step log as the run panel showed it) or **Debug** (adds exactly what was sent and what the host replied).
>   **Output style** only changes how entries are displayed, and applies to entries already recorded too.
> - **Backup & Restore**: save a copy of your app data, or restore from one. Backups contain your host list, settings, saved shell profiles, templates and skills but never your passwords or key passphrases (those stay in the **Security section** vault), so a backup file cannot leak a secret. Tick **Also include hosts CSV** to additionally write a plain spreadsheet of your hosts alongside the backup.
>   **Restore only a backup you made yourself.** A restore replaces your hosts, history, profiles, templates, skills and preferences. Some things are never taken from the file and stay as they are on this machine: custom local shells, trusted host keys, shortcut commands, the admin

> lock, and the sudo auto-fill, history clean-up, audit log and skill log location switches. Profile startup commands come back empty, and a profile that used a custom local shell comes back struck through: add the shell, then open **Edit** on the profile and save it. A host the backup leaves out, or gives a different address, port or user, loses its saved passwords, so enter them again. With an admin passcode set, unlock first. A list of what changed is shown once the app reloads, and kept in **Logs** → **Toasts History**.
>
> - **Audit log**: turn audit recording on or off.
> - **Security**: the sudo auto-fill switch, whether a connect deletes its own setup line from the host's shell history, and the admin lock. All three are described in the Security section below.
> - **Reset**: return every setting to its default, behind a confirmation.
> - **Danger Zone**: **Delete all hosts & credentials** removes every host and its stored secrets in one step. It cannot be undone, so the dialog offers to back up first (optionally including the hosts CSV) before it wipes anything.

SECTION 12

# Security

Broadside handles passwords and connects to your machines, so it is worth understanding exactly what it stores, what it sends, and the controls you have. **There is no telemetry and the app never phones home.**

### How your credentials are stored

- Secrets (SSH passwords, key passphrases and sudo passwords) are stored in the credential-manager first, which ties them to your Windows login. If that is unavailable, the app falls back to a file named credentials.age that is encrypted at rest.
- Each secret is keyed to its host, so removing a host removes its secrets. Non-secret host details (label, address, port, username, color) live in a local database in plain text.
- Once saved, a secret is never shown again. You can replace it.

### What is sent over the network

- The app makes outbound SSH connections only to the hosts you configure. **Broadcast** uses an exec-channel; **Terminals** use a full PTY. Nothing else is sent anywhere.
- Host identity is verified with TOFU: the app remembers each host's key on first connection and warns you if it changes.
- When the app fills a sudo prompt, the password is sent inside the encrypted SSH session in answer to the prompt. It is never placed on the command line or echoed back to the screen.

### The sudo auto-fill switch and admin lock

- Sudo auto-fill is a global switch in **Settings**. When on, the app answers a host's sudo password prompt using the password you stored for that host. Turn it off to always type sudo passwords yourself.
- It only answers **sudo**. Password prompts from other programs (**su**, **ssh**, a database client) are left for you, so a stored password is never sent somewhere you did not intend.
- **Keep the connect line out of the host's shell history** is **off** by default. When a terminal opens, Broadside sends the host a short setup line. Debian and Ubuntu keep that line out of the shell history by default, but many other systems do not, so the **Up** arrow finds it instead of your last command. Turn this on and Broadside deletes that one entry for you; it never touches your own commands. It has no effect on `zsh`, and applies to terminals opened after you turn it on. To see whether a host needs it, run `echo "$HISTCONTROL"` there: if the answer includes `ignorespace` or `ignoreboth`, it does not.
- The admin lock is an optional passcode that protects the credential and security controls. It is for authorization only: it uses argon2 and encrypts nothing, so losing the passcode loses no data. Setting it gives you a one-time recovery code to regain access if you forget the passcode.

> - The admin lock does not keep people who share one Windows account apart. To limit a less-trusted colleague, give them their own Windows login and a csv of hosts (which never contains passwords), so they never receive your stored secrets.

SECTION 13

# System requirements

Broadside is a Windows desktop app and runs comfortably on modest hardware.

### Minimum and recommended

| Component | Minimum | Recommended |
|---|---|---|
| Operating system | Windows 10 (64-bit), version 1809 or later | Windows 11 (64-bit), kept up to date |
| WebView2 runtime | Microsoft Edge WebView2 installed (the installer adds it if missing) | WebView2 kept current through Windows Update |
| Processor | 64-bit dual-core (x64 or ARM64) | Quad-core or better, 2.5 GHz and up |
| Memory (RAM) | 4 GB | 8 GB or more |
| Free disk space | 300 MB | 1 GB or more, for logs, saved sessions and backups |
| Display | 1280 × 720 | 1920 × 1080 or higher |
| Network | Any connection that can reach your SSH hosts | A low-latency link when broadcasting to many hosts at once |

### Worth knowing

- Broadside is Windows only: it uses ConPTY for local shells and the credential-manager for your secrets, neither of which exists on macOS or Linux.
- No administrator rights are needed to run it. WebView2 comes with Windows 11 and recent Windows 10; on an older system the installer adds it.
- Memory use grows with the number of open terminals and how much scrollback each keeps (**Settings** → **Appearance** → **Terminal scrollback**).
- Disk use grows with saved sessions, the audit log and backups, all of which you control from the **Logs** and **Settings** tabs.

### If Windows Security flags Broadside

- It can happen, and it is a false alarm: a new, unsigned app that starts shell processes is exactly what Windows Defender's automatic checks are wary of.
- A detection name ending in `!ml` (Broadside has been flagged once as `Trojan:Win32/Bearfoos.A!ml`) means no known malware was found; the file just looked unusual.
- If the app disappears, restore it from **Windows Security** under **Virus & threat protection** and **Protection history**. Before you do, check the installer's SHA-256 against the one published with the release, so you know you are restoring the file that was actually built.

> - Please report it, with the detection name and version, so it can be sent to Microsoft as a false positive. A Defender exclusion stops the warnings but also stops Defender checking that folder, so use one only as a stopgap.

SECTION 14

# Keyboard shortcuts

Every key Broadside takes for itself, and what it does. Any other key goes to whatever has the focus; in a terminal, that is the shell.

| Key | Where | What it does |
|---|---|---|
| **ANYWHERE IN BROADSIDE** | | |
| `F11` | Terminals | Maximize the selected terminal to fill the window. Press it again to restore. Esc does not restore: it goes to the program in the terminal, since editors such as vi depend on it. |
| `Ctrl+F` | Terminals, Broadcast, Logs | Open the find bar for whatever output you are looking at, instead of sending Ctrl+F to the shell. |
| `Enter` | Find bar | Go to the next match. |
| `Esc` | Find bar, dialogs, menus | Close it without doing anything. |
| **THE TERMINALS GRID** | | |
| `Ctrl+Shift+G` | Grid | Fill the screen with the grid: the sidebar, status bar, toolbar and tab strip slide away. Press it again to bring them back, or use the tab at the top of the screen. Esc does not leave fullscreen, so it still reaches vi. |
| `Esc` | Terminal controls | Hand the keyboard back to the terminal after Ctrl+Shift+T put it in the toolbar. It does not leave fullscreen. |
| `Ctrl+Shift+T` | Terminals | Put the keyboard into the toolbar, with a ring around the button that has focus. In fullscreen the hidden controls slide down too. |
| `Tab` / `Shift+Tab` | Terminal controls | Move along the toolbar and tab strip once Ctrl+Shift+T has put you there. Tab wraps round at the last control. Esc goes back to the terminal. |
| `Alt+Left` / `Alt+Right` | Terminals | Previous / next terminal tab, wrapping at the ends. Works even while you are typing in a terminal. |
| `Left` / `Right` then `Enter` | Tab strip | Move along the tabs once Tab has put the keyboard on the tab strip (Home and End jump to the ends), then Enter or Space chooses one. |
| **INSIDE A TERMINAL** | | |
| `Ctrl+Shift+C` | Terminals | Copy the selection and clear it, the same as right-clicking it. Plain Ctrl+C still interrupts what is running. |
| `Ctrl+Shift+V` | Terminals | Paste the clipboard into the shell as if typed. Plain Ctrl+V is left to the shell. |

| Key | Where | What it does |
|---|---|---|
| `Right-click` | Terminals, Skills | With text selected, copy it (a short toast confirms) and clear the highlight. With nothing selected, paste. A skill pane only copies: it never pastes into a run. |
| `Select with the mouse` | Terminals, Skills | Selects text without copying it; right-click or press Ctrl+Shift+C to copy. Hold Shift to select in tmux or vim when they have the mouse on. |
| `Ctrl+mouse wheel` | Terminals, Skills | Make the text in every terminal bigger or smaller, between 8 and 32 px. The size is saved in Settings, Appearance. Tick Strict beside that setting if a program in your terminal needs Ctrl and the wheel for itself. |
| `Tab` | Terminals | Always goes to the shell (for completion). |
| **RENAMING A TAB** | | |
| `Enter` | Tab rename | Save the new name. An empty name puts the shell's own name back. |
| `Esc` | Tab rename | Cancel and keep the old name. |
| **THE COMMAND COMPOSER (BROADCAST, PTY BROADCAST, MULTITERMINAL)** | | |
| `Enter` | Composer | Send the command. |
| `Shift+Enter` | Composer | Start a new line instead of sending, for a multi-line command. |
| `Up` / `Down` | Composer | Step back and forth through the commands you have sent. Works from the first or last line, so editing a multi-line command still moves the cursor normally. What you had typed is kept, so you can always get back to it. |
| **SFTP COMMANDER** | | |
| `Ctrl+click` | File list | Add or remove one file from the selection. |
| `Shift+click` | File list | Select everything between the last click and this one. |
| `Double-click` | File list | Open a folder or drive. A single click only selects it. |
| `Enter` | File list | Open the selected folder. |
| `Backspace` | File list | Go up one folder. |
| `Up` / `Down` | File list | Move the selection one row. |
| `F2` | File list | Rename the selected file or folder. |
| **THIS HELP PAGE** | | |
| `Type in the filter` | Help | Narrows the help to the sections that mention what you type, including these tables. Matches are marked in amber, with arrows beside the count to move between them. Punctuation is optional: ctrl shift g finds Ctrl+Shift+G. The list on the left shows how many matches each section has. |

SECTION 15

# Warnings and messages

What each message Broadside can show you means, and what to do about it.

| What you see | Where | What it means |
|---|---|---|
| ON A HOST ROW | | |
| An amber chip naming a shell (fish, csh, tcsh, …) | Hosts in full; a triangle alone in the picker rails | This host's login shell is not one Broadside can fully work with. Terminals, broadcasts, file transfer and sudo auto-fill all work as normal, but MultiTerminal cannot separate its output into command blocks and skills cannot run here. A command written for bash may not behave the same in its shell. |
| An amber “no shell” chip | Hosts in full; a triangle alone in the picker rails | What this host runs at login is not a shell but, usually, an appliance's console menu (a pfSense admin account, for example). Broadside sends it nothing on connect, so the menu appears exactly as the device draws it. MultiTerminal command blocks and skills are unavailable; terminals, broadcasts and file transfer are not affected. |
| An amber “shell unread” chip | Hosts in full; a triangle alone in the picker rails | The connection worked, but the host would not say what its login shell is (a restricted account, for example), so Broadside sends nothing rather than guess. MultiTerminal command blocks and skills are unavailable. If you know the host runs bash or zsh, open it in Hosts and set Shell integration to Always. It can also mean the host was too slow to answer: a slow network home directory, or a .bashrc that loads tools such as nvm or conda before its interactive check. The same fix applies. |
| An amber “no integration” chip | Hosts in full; a triangle alone in the picker rails | Shell integration is set to Never on this host, so Broadside sends it nothing on connect. MultiTerminal command blocks and skills are unavailable while it is set. Change it in the host's form. |
| “N checked hosts report no blocks” | MultiTerminal | Those hosts will run every command you send but show nothing here, because this page needs bash or zsh with shell integration. Nothing is lost: the output is in each host's own terminal tab. Hover the message to see which hosts, and why. |
| A red Test connection icon | Hosts | The last attempt to reach this host failed (a test, a terminal, a broadcast, an SFTP session or a skill run). It is kept after a restart and clears itself the next time anything connects to the host successfully. Hover the button for the details, and see Logs for what went wrong. |
| No credentials set | Hosts | The host is saved but has no password or key stored, so nothing can connect to it. Edit the host and add one. |
| A latency in milliseconds after a test | Hosts | Not a warning: the test connection worked, and this is how long it took to connect and log in. |
| OPENING A TERMINAL | | |

| What you see | Where | What it means |
|---|---|---|
| Unreachable: … | Terminal pane | Broadside could not reach the host; the rest of the line is Windows' explanation. Most of these are worth a Try again. |
| Unreachable: An existing connection was forcibly closed by the remote host. (os error 10054) | Terminal pane | The server accepted the connection and then hung up during login. Most often the host is refusing too many logins at once (OpenSSH's MaxStartups, 10 by default). Broadside limits and retries these, so it should be rare; if it keeps happening, raise MaxStartups on the host or open fewer terminals at a time. |
| Unreachable: connection timed out after 10s | Terminal pane | Nothing answered on that address and port. The host is off, the port is wrong, or a firewall is dropping it. Not retried automatically. |
| Authentication failed: … | Terminal pane | The host refused the credentials: wrong password, wrong username, or a key the host does not accept. A key passphrase problem looks different (see below). |
| The private key … is encrypted and no passphrase is stored for it | Terminal pane, Files, Test connection | The key file needs a passphrase and none is saved with it. Nothing was sent to the host. Edit the host, choose Replace credentials, and enter the passphrase. If you see this on a host that used to work, the key file has changed since you saved the host. |
| This key is encrypted. Enter its passphrase to save it | Host form, under Key passphrase | The key file you chose is encrypted, so a passphrase is required for it. |
| That passphrase does not open this key | Host form, under Key passphrase | The passphrase you entered does not unlock this key file. |
| No credentials stored. Edit the host on the Hosts page. | Terminal pane, Broadcast | Nothing was tried: there is no password or key saved for this host. |
| Waiting on host-key verification… | Terminal pane | The connection is held open while you answer the host-key dialog. Nothing is sent to the host until you do. |
| **HOST KEYS** | | |
| This host's key is not known yet | Host-key dialog | First connection to this host. Check the fingerprint shown against the host itself before trusting it. |
| The host key has changed: the connection was refused | Host-key dialog | The server's key is different from the one on record: either the server was rebuilt or re-keyed, or the connection is being intercepted. Nothing was sent. Do not accept it until you know which. |
| host key changed: refused | SFTP Broadcast | The same thing, on one host of a batch. The others carried on. |
| **WHILE A TERMINAL IS OPEN** | | |
| A slim banner across the bottom of a tile | Terminal pane | The shell has exited or the connection dropped. The output stays readable; Reconnect starts a fresh shell, Close tab closes it. |

| What you see | Where | What it means |
|---|---|---|
| A pill on that banner reading “SSH 12/14 86%” | Terminal pane, remote sessions only | While the pane sits closed, Broadside keeps checking whether the host's SSH port is answering, so you can see when a rebooted machine is back. Green: answering now, so Reconnect should work. Red: not answering. Amber: missed the last check but has been answering. The figures cover the last two minutes. How often it checks is set on the host's form. It never logs in, and it stops when you reconnect or close the tab. |
| An amber line across the bottom of the pane, with “Got it” | Terminal pane | The same warning as the amber chip on the host row, shown once where you are working. Nothing is broken and the tab is usable. Got it hides it for that host until something about the host's shell changes. |
| Every terminal is hidden from the grid. | Grid | Every open terminal has been taken out of the grid. Use the grid button on a tab, or Unhide all in the Sessions menu, to bring them back. |

Anything that reaches the **Logs** tab is recorded with the host and what was running, so look there for a message that went by too quickly.

SECTION 16

# Glossary

Plain-language definitions for the technical terms used above. Links throughout the documentation jump here.

**SSH (Secure Shell)**

A network protocol for running commands on another computer over an encrypted connection. Broadside uses SSH to reach every host you add.

**SFTP (SSH File Transfer Protocol)**

A way to browse and copy files over an SSH connection, using the same login and encrypted channel as a remote shell. Broadside's SFTP tab uses it to move files to and from your hosts.

**PTY (pseudo-terminal)**

A two-way text channel that behaves like a real terminal screen and keyboard. Interactive programs (a login shell, a text editor, a progress bar) need a PTY so they can redraw the screen and read keystrokes. Each Terminals tab opens its own PTY.

**ConPTY (Windows pseudo-console)**

Windows' built-in version of a PTY. Broadside uses it to run local Windows shells (PowerShell, Command Prompt, WSL) inside a Terminals tab, the same way it runs remote shells over SSH.

**Shell**

The program that reads the commands you type and runs them, for example bash on Linux or PowerShell on Windows. A terminal tab is a window onto a shell.

**POSIX**

The standard that Unix-like systems follow, which is why a command written for Linux usually works on macOS or inside WSL. In Broadside, Bash-like terminals such as Git Bash, MSYS2 and Cygwin count as POSIX: they are tagged `posix` and join the **WSL / POSIX** tab in **PTY Broadcast**.

**sudo**

A Unix command that runs another command with administrator rights. It usually asks for your password first. Broadside can fill that password prompt for you when the feature is turned on (see the Security section).

**TOFU and host keys**

Every SSH server has a unique host key, a fingerprint that proves it is the same machine you connected to before. TOFU (Trust On First Use) means the app remembers a host's key the first time you connect and warns you if it ever changes, which can signal that the connection is being intercepted.

### SSH key vs password

Two ways to prove who you are to an SSH server. A password is a secret you type. An SSH key is a pair of files (a private key you keep and a public key the server holds); the private key can be protected by a passphrase. Keys are generally more secure than passwords.

### Key passphrase

A secret that unlocks an SSH private key file, so that a copy of the file alone is useless. It is only used on your own machine and never sent to the host. Broadside stores it with your other secrets.

### Windows Credential Manager

A secrets vault built into Windows, tied to your Windows user account. Broadside stores passwords and passphrases here first so they are protected by your Windows login and never kept in plain text.

### argon2

A modern password-hashing algorithm. Hashing turns a passcode into a fixed fingerprint that cannot be reversed back into the original. The admin lock stores only this fingerprint, so the passcode itself is never saved anywhere.

### CSV

A plain-text spreadsheet format (comma-separated values) where each line is one record. Broadside can import a list of hosts from a CSV file. The importer never reads passwords from the file.

### Exec channel

A way to run a single command on a host over SSH and collect its output, without opening a full interactive screen. Broadcast uses this. It is faster for one-shot commands but cannot drive interactive programs.